

# Voice-Based Concept Understanding Analyser (VBCUA)

## Project Description:

Voice-Based Concept Understanding Analyser (VBCUA) is an AI-powered application designed to evaluate how well a user explains a technical concept through spoken audio. The platform transcribes the recording, compares it against a reference concept, analyzes speech fluency, and generates a downloadable PDF report.

Built using Streamlit, OpenAI Whisper, Sentence-BERT, and Librosa, the application enables students, educators, and trainers to objectively assess spoken conceptual understanding. The system also generates waveform visualizations and maintains results for the duration of the session for future review.

## Scenarios

### Scenario 1:

A student uploads an audio explanation for a concept such as Machine Learning. The system transcribes the speech using Whisper and compares it with a reference concept using Sentence-BERT, generating a conceptual understanding score along with qualitative feedback.

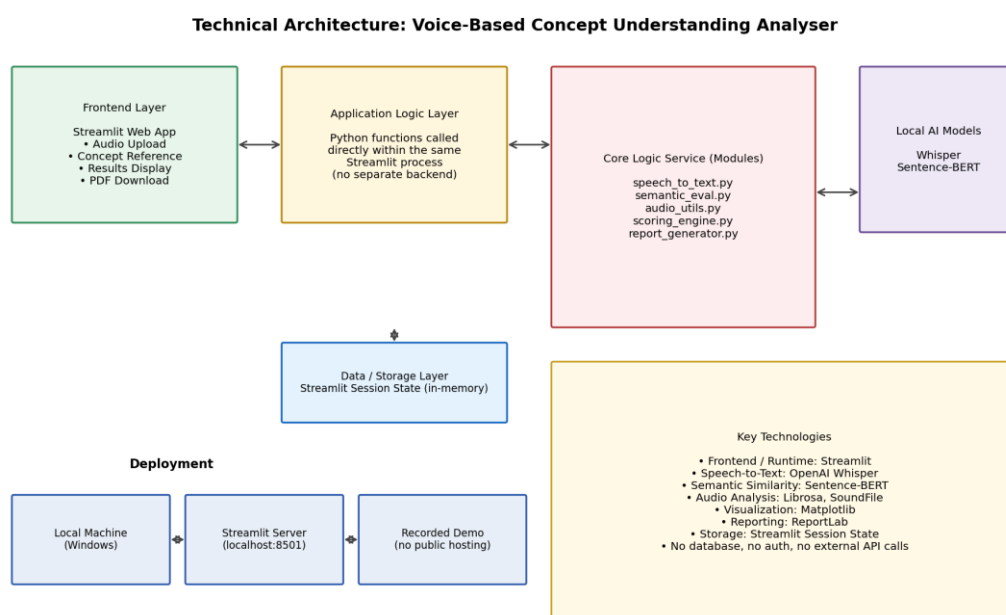
### Scenario 2:

A learner practicing interview preparation records a spoken explanation of a topic. The platform evaluates fluency-related metrics including filler word usage, pause ratio, and audio energy using Librosa-based signal analysis.

### Scenario 3:

After analysis, the platform displays the transcript, similarity score, waveform, and final comprehension score within an interactive Streamlit dashboard, and generates a downloadable PDF report for future review.

## Technical Architecture



**Description:**

VBCUA follows a modular single-process architecture.

- The Frontend Layer consists of a Streamlit-based user interface.
- The Application Logic Layer calls Python functions directly within the same process — there is no separate backend service.
- The Core Logic Service contains modules for speech-to-text, semantic similarity, audio feature extraction, scoring, and PDF report generation.
- The Data/Storage Layer uses Streamlit Session State (in-memory) rather than a database.

The architecture intentionally avoids unnecessary complexity — no authentication service, no database, and no external API calls — since VBCUA is a single-user, locally run tool.

## Pre-requisites

- Python Programming Knowledge
- Streamlit Framework Knowledge
- OpenAI Whisper Familiarity
- Sentence-BERT / Sentence-Transformers Concepts
- Librosa for Audio Processing
- ReportLab for PDF Generation
- Git and GitHub Basics
- Virtual Environment Management
- ffmpeg installed on the system (required by Whisper)

---

## Project Workflow

**Milestone 1: Environment Setup and Dependency Configuration**

**Milestone 2: Core Logic Development (Speech Understanding & Evaluation Engine)**

**Milestone 3: Streamlit UI Implementation**

**Milestone 4: Testing and Validation**

**Milestone 5: Deployment and Documentation**

## Milestone 1: Environment Setup and Dependency Configuration

**Activity 1.1: Define Project Requirements**

- Identify evaluation requirements.
- Define spoken-concept assessment use cases.
- Finalize project scope.

**Activity 1.2: Design System Architecture**

- Create the architecture diagram.
- Define interactions between UI, core logic modules, and session state.

**Activity 1.3: Set Up Development Environment**

- Install Python 3.10+ and ffmpeg.
- Create and activate a virtual environment.
- Install dependencies from requirements.txt.

**Activity 1.4: Create Project Structure**

Create files for:

- app.py
- speech\_to\_text.py
- audio\_utils.py
- semantic\_eval.py
- scoring\_engine.py
- report\_generator.py
- requirements.txt

---

## Milestone 2: Core Logic Development (Speech Understanding & Evaluation Engine)

### Activity 2.1: Develop Speech-to-Text Module

Integrate OpenAI Whisper to convert uploaded audio into text.

*Functions:* `speech_to_text()`

Responsibilities:

- Load and cache the Whisper model.
- Transcribe uploaded audio to plain text.

### Activity 2.2: Develop Semantic Similarity Module

Integrate Sentence-BERT to compare the transcript against a reference concept.

*Functions:* `compute_similarity()`, `get_reference_text()`

Responsibilities:

- Generate embeddings for the transcript and reference concept.
- Compute and normalize cosine similarity (0–1 scale).

### Activity 2.3: Develop Audio Feature Extraction Module

Implement fluency analysis using Librosa.

*Functions:* `extract_audio_features()`, `filler_word_ratio()`

Responsibilities:

- Compute pause ratio and RMS energy.
- Generate the waveform plot.
- Calculate filler-word ratio from the transcript.

### Activity 2.4: Develop the Scoring Engine

*Functions:* `evaluate_understanding()`

Responsibilities:

- Combine similarity, filler ratio, pause ratio, and energy into a final score.
- Classify the result as Strong / Moderate / Poor Understanding.

### Activity 2.5: Develop the PDF Report Generator

*Functions:* `generate_pdf_report()`

Responsibilities:

- Assemble the reference concept, transcript, waveform, and evaluation summary into a PDF using ReportLab.

## Milestone 3: Streamlit UI Implementation

### Activity 3.1: Develop the Streamlit Interface

Create a single-page interface for:

- Audio upload.
- Concept reference selection.
- Results display.
- PDF report download.

### Activity 3.2: Add User Interaction Features

Implement:

- Session-state management to persist results during the active session.
  - Waveform display.
  - Evaluation summary table.
  - PDF download button.
- 

## Milestone 4: Testing and Validation

### Activity 4.1: Functional Testing

The scoring engine was verified against a known worked example: similarity = 0.03, filler ratio = 0.03, pause ratio = 0.11, RMS energy = 0.1633 → produced Final Score = 60/100, Understanding Level = “Moderate Understanding,” confirming the formula matches the expected output exactly.

### Activity 4.2: Performance Testing

Audio feature extraction was measured directly: ~2.13 seconds on a cold start (one-time library warmup) and ~0.5 seconds on subsequent calls within the same session.

### Activity 4.3: Environment/Dependency Testing

The application was validated on a clean virtual environment containing only the 7 source files and requirements.txt, confirming the project is self-contained and reproducible.

---

## Milestone 5: Deployment and Documentation

### Activity 5.1: Local Deployment

Run Streamlit:

```
streamlit run app.py
```

### Activity 5.2: Version Control

Upload project to GitHub. Repository includes:

- Source code.
  - Documentation (all phase folders).
  - Screenshots.
  - Demo video link.
  - README.
-

## Exploring the Application Pages

### Home Page

Description:

The home page allows users to:

- Upload a spoken audio explanation.
- Select a reference concept.
- Trigger the analysis.

### Results Section

Description:

Displays the transcribed explanation, semantic similarity score, filler word ratio, waveform visualization, and final understanding score.

### PDF Report

Description:

A downloadable report containing the same evaluation summary shown on screen, for offline review.

---

## Conclusion

VBCUA successfully demonstrates how AI-driven speech and language analysis can improve the way spoken conceptual understanding is evaluated. The system assists students and educators by generating objective understanding scores, detecting fluency issues, and maintaining a structured PDF record of each evaluation.

The modular architecture built using Streamlit, Whisper, and Sentence-BERT ensures maintainability without unnecessary complexity. Future enhancements include persistent storage, multilingual support, an expanded reference-concept bank, and real-time voice input for immediate feedback.

This project highlights the practical application of AI in making spoken self-assessment more objective, accessible, and useful for learners.